Yuxuan Lin — Site Framework

A small, static, hand-coded website for Yuxuan Lin: composer, sound artist, and creative technologist. Designed as a *framework* — every section is editable with no build step, no CMS, no dependencies. Just HTML, CSS, and a single JavaScript file.

```
yuxuan-lin-site/
├── index.html          ← landing
├── works.html          ← catalogue of compositions
├── performances.html   ← upcoming + archive
├── bio.html            ← short / medium / long versions
├── store.html          ← self-published scores + cart
├── projects.html       ← music + code projects (with demo & README modals)
├── contact.html        ← contact form
├── styles.css          ← all visual design lives here
├── main.js             ← all behavior lives here
└── assets/             ← put images, audio, PDFs here
```

Quick start

To preview locally, run any static server from the project root:

bash

```bash
# Python
python3 -m http.server 8000

# Node
npx serve .
```

Open `http://localhost:8000`. No build step. Edit any `.html` file and refresh.

How to add or edit content

Each page is built from clearly-marked, repeated blocks. Find the relevant section in the HTML file and copy an existing block, then change the text.

Add a new work (`works.html`)

Find the `<div class="works-list">` and copy any `.work-row`:

html

```
<div class="work-row" data-category="ensemble">
  <div class="work-year">2026</div>
  <div class="work-title">Title</div>
  <div class="work-meta">Description, dedicatee, premiere context.</div>
  <div class="work-instr">Instrumentation</div>
  <div class="work-duration">~ 12′</div>
</div>
```

`data-category` controls which filter button it shows under. Valid values: `solo`, `chamber`, `ensemble`, `electronic`, `installation`, `vocal`. Add more categories by extending the filter bar at the top of the file.

Add a performance (`performances.html`)

Copy any `.perf-row`. Use `class="perf-row upcoming"` for upcoming dates (this highlights the date in crimson and shows a colored tag). For past dates, use `class="perf-row"`.

Edit bio text (`bio.html`)

Three `<article class="bio-version">` blocks for short / medium / long. Edit the text directly. The "Copy" button next to each version automatically copies that version's text — no extra wiring needed.

Add a score to the store (`store.html`)

Copy any `<article class="score-card">`. Required `data-*` attributes:

```html
<article class="score-card"
  data-id="unique-id"
  data-title="The Title"
  data-price="22">
  ...
</article>
```

The cart reads these attributes when the "Add to cart" button is clicked.

Add a project (`projects.html`)

Copy any `<article class="project-card">`. Use `class="project-card code"` for code projects (changes the tag dot color). The modal pulls four attributes:

- `data-title` — project title
- `data-kind` — tag line (e.g. "Code · Library · 2024")
- `data-description` — paragraph below the title
- `data-demo` — URL to embed. Auto-detected:
  - `.mp3 / .wav / .ogg` → `<audio>`
  - `.mp4 / .webm` → `<video>`

    - anything else → `<iframe>` (YouTube, Vimeo, p5 sketches, etc.)
- `data-readme` — multiline README text. Use `## Heading` for section headers and backticks for `code`. Newlines are preserved.

Leave `data-demo` empty to show the placeholder pattern.

Wiring up real functionality

The framework includes working JavaScript stubs for the cart and contact form. They're intentionally local-only so you can preview without backend setup. To make them production-ready:

Contact form

The form currently logs submissions to the browser console. Three easy options:

Option A — Formspree (recommended, free tier available):

1. Sign up at https://formspree.io and create a form. You'll get an endpoint URL.
2. In `contact.html`, change the form opening tag to:

```
html

<form class="contact-form" action="https://formspree.io/f/YOUR_ID" method="P
```

3. In `main.js`, find `initContactForm` and remove the `e.preventDefault()` line so the form submits normally. Or keep the JS handler and `fetch()` to the endpoint for inline status messages.

Option B — Netlify Forms (free if you host on Netlify):

Add `netlify` and `name="contact"` to the form tag:

```
html

<form class="contact-form" name="contact" netlify>
```

Netlify auto-detects and routes submissions to a dashboard.

Option C — Your own backend: point the form's `action` to your endpoint.

Store / cart

The cart is fully functional in-memory. The "Checkout" button currently shows an alert. Three options to make it real:

Option A — Stripe Checkout (smoothest):

1. Create a Stripe account, create a Product for each score, copy each Product's `Price ID`.
2. Add a `data-stripe-price` attribute to each `.score-card`:

```html
<article class="score-card" data-id="..." data-stripe-price="price_xxx" ...>
```

3. In `main.js`, in the checkout handler, build a Checkout Session via Stripe's client-side redirect:

```js
const stripe = Stripe('pk_live_...');
stripe.redirectToCheckout({
  lineItems: cart.items().map(i => ({
    price: i.stripePrice,
    quantity: i.qty
  })),
  mode: 'payment',
  successUrl: 'https://yuxuanlin.com/thanks.html',
  cancelUrl: 'https://yuxuanlin.com/store.html'
});
```

4. Stripe handles PDF delivery via its "Customer Portal" or via webhooks that email a download link.

Option B — Gumroad (simpler, less control):

Replace each "Add to cart" button with a Gumroad overlay button. Gumroad hosts the PDFs and handles delivery. The current cart UI can be removed entirely if you go this route.

Option C — Bandcamp / Sellfy: Similar to Gumroad. Best if Yuxuan already sells music there.

Adding a newsletter

The footer has a stub `Newsletter` link. Easiest options:

- Substack — link to her Substack URL.
- Buttondown — embed their form (one `<script>` tag).
- Mailchimp / ConvertKit — same.

Hosting

This is a static site, so it works anywhere:

- **Netlify** (recommended): drag the folder into netlify.app, get a URL
  - free SSL in 30 seconds. Connects to GitHub for auto-deploy on edits.
- **Cloudflare Pages**, **Vercel**, **GitHub Pages**: all equally fine.
- **Traditional host**: just FTP the folder up. No Node, no Python, no database needed.

After deploying to Netlify (or similar):

1. Buy `yuxuanlin.com` (or whatever) at Namecheap, Porkbun, etc.
2. Point its DNS to the host (the host gives you the exact records).
3. Done.

Aesthetic notes

The design is built around a single editorial idea — a printed archive on warm paper, with a faint waveform underneath. Colors and typography are centralized as CSS variables at the top of `styles.css`:

```css
--paper: #ece6d8;
--ink: #161412;
--accent: #8a2418;
--font-display: 'Fraunces', serif;
--font-serif: 'Cormorant Garamond', serif;
--font-mono: 'JetBrains Mono', monospace;
```

Change those six values and the entire site re-themes. No other edits needed.

Accessibility & SEO checklist

The framework includes the basics — semantic HTML, alt text where applicable, keyboard-navigable buttons, focus styles, and a sensible heading hierarchy. Before launching, recommended additions:

- Add an `og:image` and `twitter:card` meta block per page (in `<head>`)
- Add a real `favicon.ico` / `apple-touch-icon.png` to the root
- Add `<meta name="description">` to each page (template included on index)
- Add a `sitemap.xml` and `robots.txt` (most hosts generate these for you)

License

Questions about the framework: see comments in `main.js` and `styles.css`. Both are written to be read.